CALIFORNIA STATE UNIVERSITY
LONG BEACH

CECS 341: Computer Architecture & Organization

Hossein Sayadi, PhD

Department of Computer Engineering & Computer Science
California State University, Long Beach

Fall 2025

Lecture Outline

- MIPS Assembly to Machine Language Translation
- Program Execution
  - Programming languages (high-level, assembly, machine)
  - Compiler, assembler, linker, and loader
  - Instruction cycle

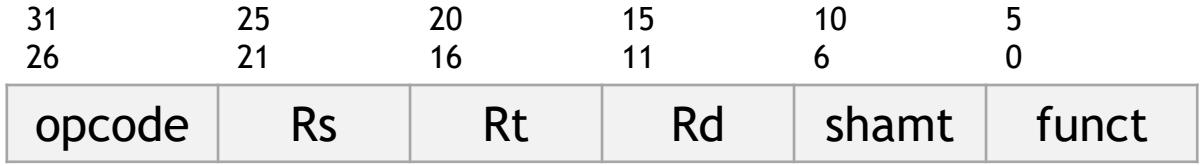
MIPS Assembly to Machine Language

Representing Instructions

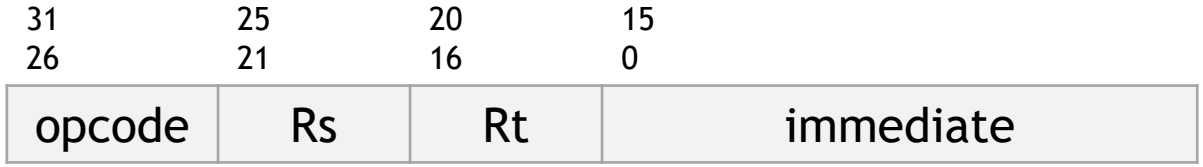
- Instructions are encoded in binary
  - Called machine code
- MIPS instructions
  - Encoded as 32-bit instruction words
  - Small number of formats encoding operation code (opcode), register numbers, …
- Register numbers
  - $t0 – $t7 are reg's 8 – 15
  - $t8 – $t9 are reg's 24 – 25
  - $s0 – $s7 are reg's 16 – 23

MIPS Machine Language

- **Assembling instructions:** Use the instruction formats (R, I, and J) to translate assembly language instructions into machine language



R-type



I-type

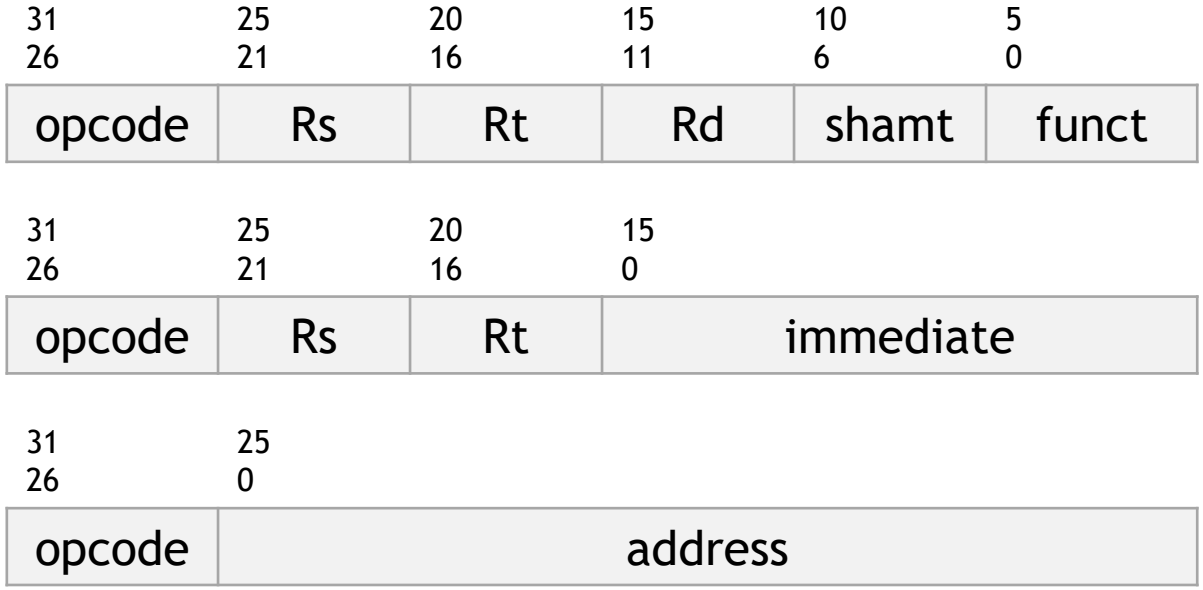


J-type

Component	Description
op, funct	codes that determine operation to perform
rs, rt, rd	register numbers for args and destination
shamt, imm, addr	values embedded in the instruction

MIPS Machine Language

- **Assembling instructions:** Use the instruction formats (R, I, and J) to translate assembly language instructions into machine language

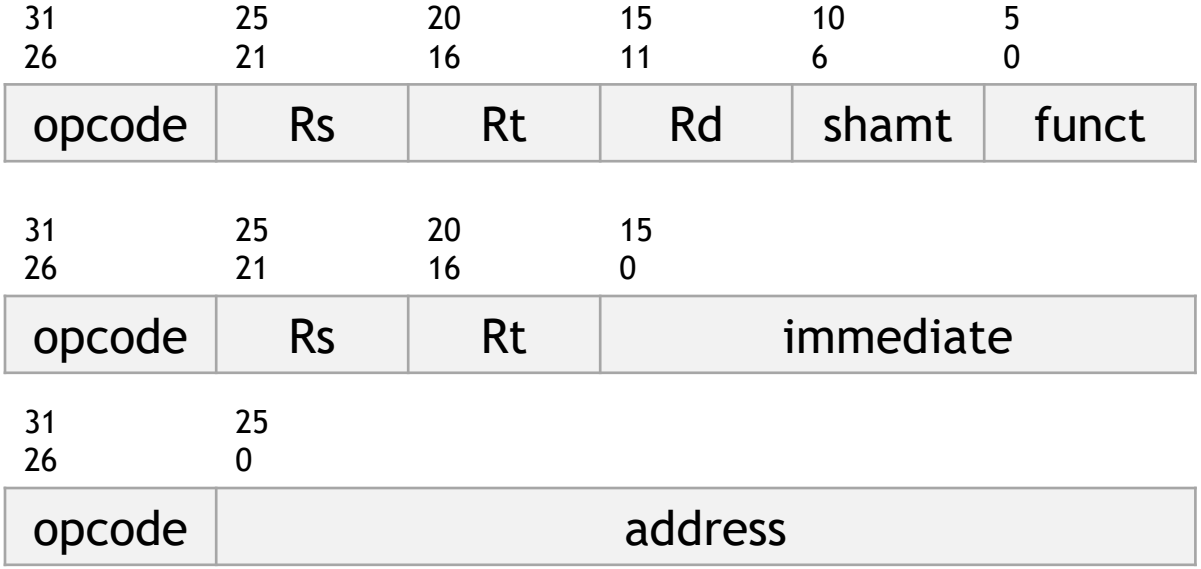


R-type: Opcode bits [31 – 26] + funct bits [5 – 0] are used to specify the instr.

I-type and **J**-type: Opcode bits [31 – 26] are used to specify the instruction

MIPS Machine Language

- **Assembling instructions:** Use the instruction formats (R, I, and J) to translate assembly language instructions into machine language



R-type

I-type

J-type

How to assemble a single instruction

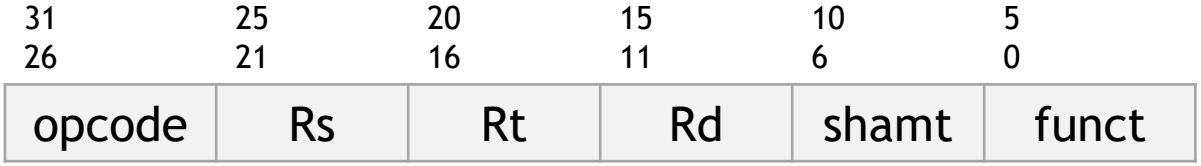
1. decide which instruction format it is (R, I, J)
2. determine value of each component
3. convert to binary
4. convert to hexadecimal

MIPS Addressing Modes

- Register addressing (R-Type)
  - Operand is stored in a register
- Base or displacement addressing (I-Type)
  - Operand at the memory location specified by a register value plus a displacement given in the instruction;
    - e.g. lw, $t0, 25($s0)
- Immediate addressing (I-Type)
  - Operand is a constant within the instruction itself
- PC-relative addressing (I-Type)
  - The address is the sum of the PC and a constant in the instruction
- Pseudo-direct addressing (J-type)
  - New PC = {(upper 4 bits of PC+4), 26-bit constant, 00}

MIPS Machine Language

- **Assembling instructions:** Use the instruction formats (R, I, and J) to translate assembly language instructions into machine language

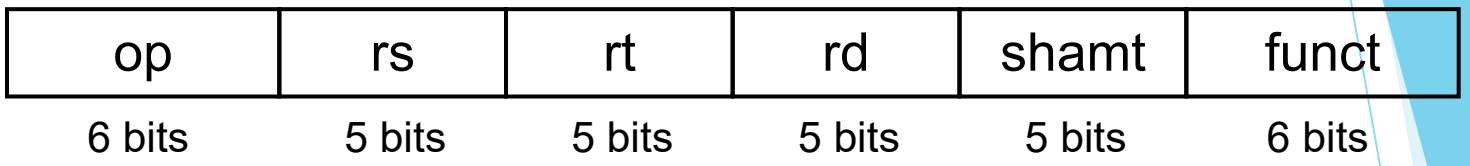


R-type

R-type instruction

• op	6 bits	always zero!
• rs	5 bits	1st argument register
• rt	5 bits	2nd argument register
• rd	5 bits	destination register
• shamt	5 bits	used in shift instructions (for us, always 0s)
• funct	6 bits	code for the operation to perform
	32 bits	

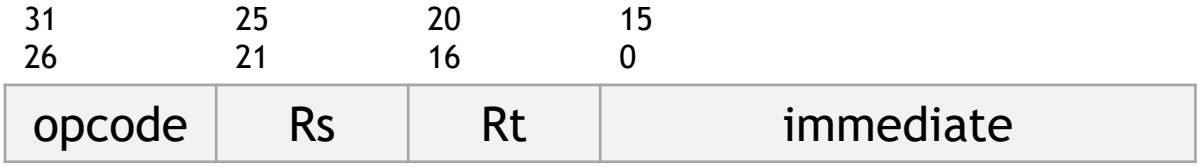
Shift Operations



- shamt: how many positions to shift
- Shift left logical
  - Shift left and fill with 0 bits
  - `sll` by *i* bits multiplies by 2^i
- Shift right logical
  - Shift right and fill with 0 bits
  - `srl` by *i* bits divides by 2^i (unsigned only)

MIPS Machine Language

- **Assembling instructions:** Use the instruction formats (R, I, and J) to translate assembly language instructions into machine language



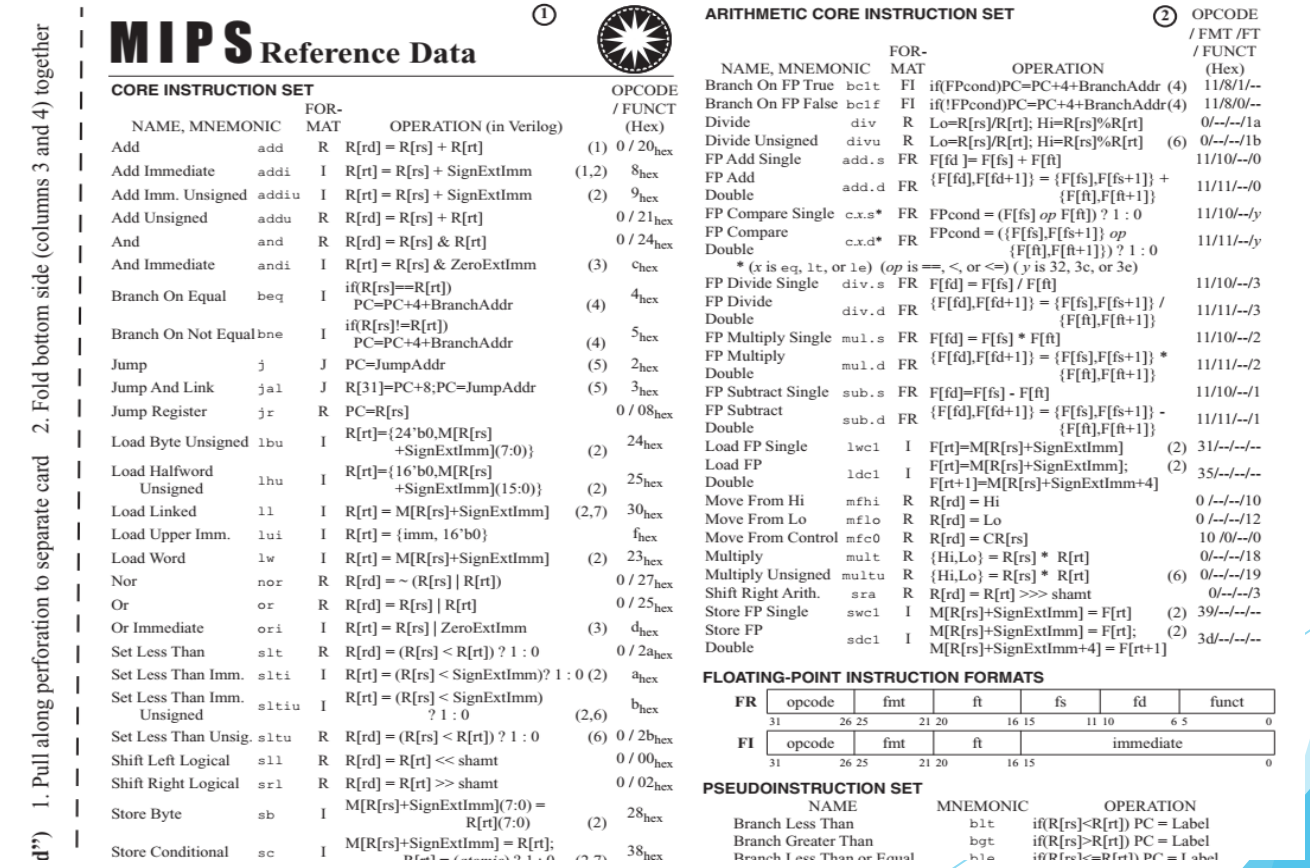
I-type

I-type instruction

• op	6 bits	code for the operation to perform
• rs	5 bits	1st argument register
• rt	5 bits	destination or 2nd argument register
• imm	16 bits	constant value embedded in instruction
	32 bits	

MIPS Machine Language

- **MIPS reference data**
  - Available on BeachBoard



1. Pull along perforation to separate card 2. Fold bottom side (columns 3 and 4) together

MIPS Reference Data

①

CORE INSTRUCTION SET

NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)		OPCODE / FUNCT (Hex)
Add	add	R	R[rd] = R[rs] + R[rt]	(1)	0 / 20_{hex}
Add Immediate	addi	I	R[rt] = R[rs] + SignExtImm	(1,2)	8_{hex}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + SignExtImm	(2)	9_{hex}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]		0 / 21_{hex}
And	and	R	R[rd] = R[rs] & R[rt]		0 / 24_{hex}
And Immediate	andi	I	R[rt] = R[rs] & ZeroExtImm	(3)	c_{hex}
Branch On Equal	beq	I	if(R[rs]==R[rt]) PC=PC+4+BranchAddr	(4)	4_{hex}
Branch On Not Equal	bne	I	if(R[rs]!=R[rt]) PC=PC+4+BranchAddr	(4)	5_{hex}
Jump	j	J	PC=JumpAddr	(5)	2_{hex}
Jump And Link	jal	J	R[31]=PC+8;PC=JumpAddr	(5)	3_{hex}
Jump Register	jr	R	PC=R[rs]		0 / 08_{hex}
Load Byte Unsigned	lbu	I	R[rt]={24'b0,M[R[rs]+SignExtImm](7:0)}	(2)	24_{hex}
Load Halfword Unsigned	lhu	I	R[rt]={16'b0,M[R[rs]+SignExtImm](15:0)}	(2)	25_{hex}
Load Linked	ll	I	R[rt] = M[R[rs]+SignExtImm]	(2,7)	30_{hex}
Load Upper Imm.	lui	I	R[rt] = {imm, 16'b0}		f_{hex}
Load Word	lw	I	R[rt] = M[R[rs]+SignExtImm]	(2)	23_{hex}
Nor	nor	R	R[rd] = ~ (R[rs] | R[rt])		0 / 27_{hex}
Or	or	R	R[rd] = R[rs] | R[rt]		0 / 25_{hex}
Or Immediate	ori	I	R[rt] = R[rs] | ZeroExtImm	(3)	d_{hex}
Set Less Than	slt	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0		0 / $2a_{hex}$
Set Less Than Imm.	slti	I	R[rt] = (R[rs] < SignExtImm)? 1 : 0	(2)	a_{hex}
Set Less Than Imm. Unsigned	sltiu	I	R[rt] = (R[rs] < SignExtImm) ? 1 : 0	(2,6)	b_{hex}
Set Less Than Unsig.	sltu	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0	(6)	0 / $2b_{hex}$
Shift Left Logical	sll	R	R[rd] = R[rt] << shamt		0 / 00_{hex}
Shift Right Logical	srl	R	R[rd] = R[rt] >> shamt		0 / 02_{hex}
Store Byte	sb	I	M[R[rs]+SignExtImm](7:0) = R[rt](7:0)	(2)	28_{hex}
Store Conditional	sc	I	M[R[rs]+SignExtImm] = R[rt];	(2,7)	38_{hex}

ARITHMETIC CORE INSTRUCTION SET

②

NAME, MNEMONIC		FOR-MAT	OPERATION		OPCODE / FMT /FT / FUNCT (Hex)
Branch On FP True	bc1t	FI	if(FPcond)PC=PC+4+BranchAddr	(4)	11/8/1/--
Branch On FP False	bc1f	FI	if(!FPcond)PC=PC+4+BranchAddr	(4)	11/8/0/--
Divide	div	R	Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]		0/--/--/1a
Divide Unsigned	divu	R	Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]	(6)	0/--/--/1b
FP Add Single	add.s	FR	F[fd]= F[fs] + F[ft]		11/10/--/0
FP Add Double	add.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} + {F[ft],F[ft+1]}		11/11/--/0
FP Compare Single	c.x.s*	FR	FPcond = (F[fs] *op* F[ft]) ? 1 : 0		11/10/--/*y*
FP Compare Double	c.x.d*	FR	FPcond = ({F[fs],F[fs+1]} *op* {F[ft],F[ft+1]}) ? 1 : 0		11/11/--/*y*
* (*x* is eq, lt, or le) (*op* is ==, <, or <=) (*y* is 32, 3c, or 3e)					
FP Divide Single	div.s	FR	F[fd] = F[fs] / F[ft]		11/10/--/3
FP Divide Double	div.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} / {F[ft],F[ft+1]}		11/11/--/3
FP Multiply Single	mul.s	FR	F[fd] = F[fs] * F[ft]		11/10/--/2
FP Multiply Double	mul.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} * {F[ft],F[ft+1]}		11/11/--/2
FP Subtract Single	sub.s	FR	F[fd]=F[fs] - F[ft]		11/10/--/1
FP Subtract Double	sub.d	FR	{F[fd],F[fd+1]} = {F[fs],F[fs+1]} - {F[ft],F[ft+1]}		11/11/--/1
Load FP Single	lwc1	I	F[rt]=M[R[rs]+SignExtImm]	(2)	31/--/--/--
Load FP Double	ldc1	I	F[rt]=M[R[rs]+SignExtImm]; F[rt+1]=M[R[rs]+SignExtImm+4]	(2)	35/--/--/--
Move From Hi	mfhi	R	R[rd] = Hi		0 /--/--/10
Move From Lo	mflo	R	R[rd] = Lo		0 /--/--/12
Move From Control	mfc0	R	R[rd] = CR[rs]		10 /0/--/0
Multiply	mult	R	{Hi,Lo} = R[rs] * R[rt]		0/--/--/18
Multiply Unsigned	multu	R	{Hi,Lo} = R[rs] * R[rt]	(6)	0/--/--/19
Shift Right Arith.	sra	R	R[rd] = R[rt] >>> shamt		0/--/--/3
Store FP Single	swc1	I	M[R[rs]+SignExtImm] = F[rt]	(2)	39/--/--/--
Store FP Double	sdc1	I	M[R[rs]+SignExtImm] = F[rt]; M[R[rs]+SignExtImm+4] = F[rt+1]	(2)	3d/--/--/--

FLOATING-POINT INSTRUCTION FORMATS

FR	opcode	fmt	ft	fs	fd	funct
	31 26	25 21	20 16	15 11	10 6	5 0
FI	opcode	fmt	ft	immediate		
	31 26	25 21	20 16	15 0		

PSEUDOINSTRUCTION SET

NAME	MNEMONIC	OPERATION
Branch Less Than	blt	if(R[rs]<R[rt]) PC = Label
Branch Greater Than	bgt	if(R[rs]>R[rt]) PC = Label
Branch Less Than or Equal	ble	if(R[rs]<=R[rt]) PC = Label

MIPS Machine Language

- **MIPS reference data**

REGISTER NAME, NUMBER, USE, CALL CONVENTION

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
$zero	0	The Constant Value 0	N.A.
$at	1	Assembler Temporary	No
$v0-$v1	2-3	Values for Function Results and Expression Evaluation	No
$a0-$a3	4-7	Arguments	No
$t0-$t7	8-15	Temporaries	No
$s0-$s7	16-23	Saved Temporaries	Yes
$t8-$t9	24-25	Temporaries	No
$k0-$k1	26-27	Reserved for OS Kernel	No
$gp	28	Global Pointer	Yes
$sp	29	Stack Pointer	Yes
$fp	30	Frame Pointer	Yes
$ra	31	Return Address	No

Example: R-type Instruction

- Convert the given MIPS AL instruction to a MIPS ML instruction.

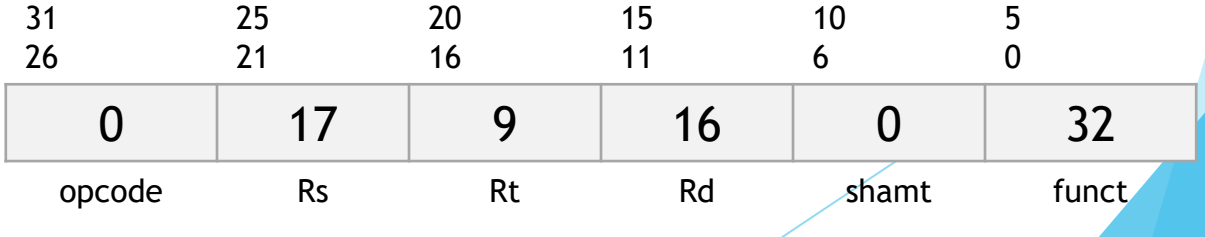
```
add $s0, $s1, $t1
```

Instruction type: R
(see MIPS Reference Card)

CORE INSTRUCTION SET

NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)		OPCODE / FUNCT (Hex)
Add	add	R	R[rd] = R[rs] + R[rt]	(1)	0 / 20_{hex}

$t0-$t7	8-15	Temporaries	No
$s0-$s7	16-23	Saved Temporaries	Yes



Example: R-type Instruction

- Convert the given MIPS AL instruction to a MIPS ML instruction.

```
add $s0, $s1, $t1
```

Instruction type: R
(see MIPS Reference Card)

31–26	25–21	20–16	15–11	10–6	5–0
0	17	9	16	0	32
opcode	Rs	Rt	Rd	shamt	funct

31–26	25–21	20–16	15–11	10–6	5–0
000000	10001	01001	10000	00000	100000

0000 0010 0010 1001 1000 0000 0010 0000 (binary)

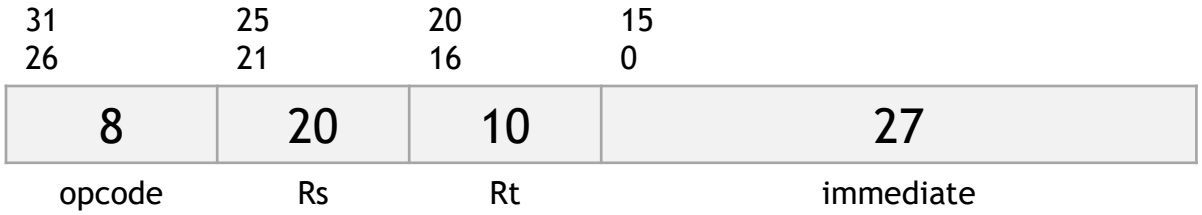
0x02298020 (hexadecimal)

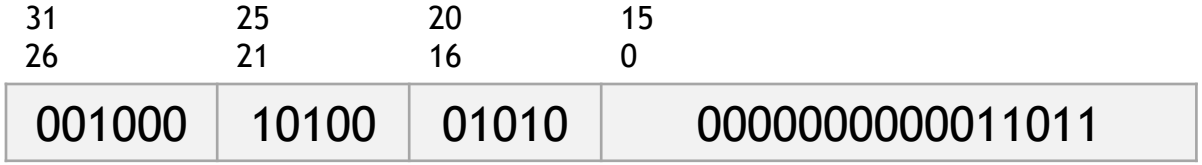
Example: I-type Instruction

- Convert the given MIPS AL instruction to a MIPS ML instruction.

```
addi $t2, $s4, 27
```

Instruction type: I
(see MIPS Reference Card)





0010 0010 1000 1010 0000 0000 0001 1011 (binary)

0x228A001B (hexadecimal)

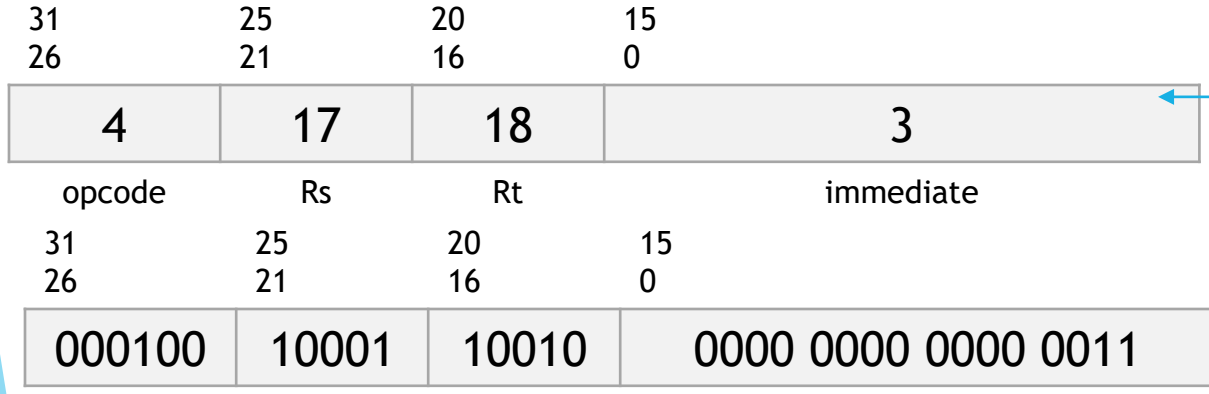
Example: Branch Instruction

- Convert the MIPS **branch** instruction to a MIPS ML instruction.

```
0x1000          beq $s1, $s2, Next
0x1004          …
0x1008          …
0x100C          …
0x1010  Next:   add $t0, $t1, $t2
```

Instruction type: I
(see MIPS Reference Card)

BranchAddr = PC + 4 + (immediate << 2)
= 0x1000 + 4 + (3 * 4)
= 0x1010



Immediate field:
no. of instructions to add to (or subtract from) the PC, starting at the instruction following the branch

(binary)

0x12320003

(hexadecimal)

Branch Addressing

- Branch instructions specify
  - Opcode, two registers, target address
- Most branch targets are near branch
  - Forward or backward



- PC-relative addressing
  - Target address = PC + offset × 4
  - PC already incremented by 4 by this time

Example: Jump Instruction

- Convert the MIPS **jump** instruction to a MIPS ML instruction.

```
0x40000000                  j       Label
        …                           …
0x400000A4          Label:  Nop
        …                           …
0x4004C100                  j       Label
```

How to calculate the address component of Jump:

- Get Address at label in Hex — 0x400000A4
- Drop the first Hex digit — 0x00000A4 = 0xA4
- Convert to binary — 10100100
- Drop the last two bits — 101001

Example: Jump Instruction

- Convert the MIPS **jump** instruction to a MIPS ML instruction.

```
0x40000000          j       Label
        …                   …
0x400000A4    Label:  Nop
        …                   …
0x4004C100          j       Label
```

op	target address

op = 2 (look up opcode for `j`)
addr = `101001` (from previous slide)

2	`101001`

`0000 10`	`00 0000 0000 0000 0000 0010 1001`

`0x08000029`

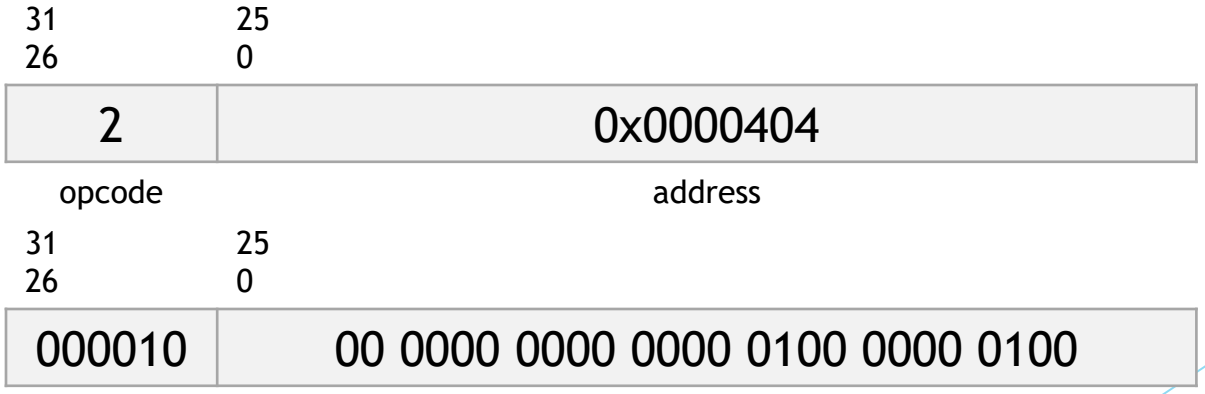
Example: Jump Instruction

- Convert the MIPS **jump** instruction to a MIPS ML instruction.

```
0x1000          j Next
0x1004          …
0x1008          …
0x100C          …
0x1010  Next: add $t0, $t1, $t2
```

Instruction type: J
(see MIPS Reference Card)

JumpAddr = (PC + 4)[31:28] : address : 00
= 0000 : 0x0000404 : 00
= 0x00001010

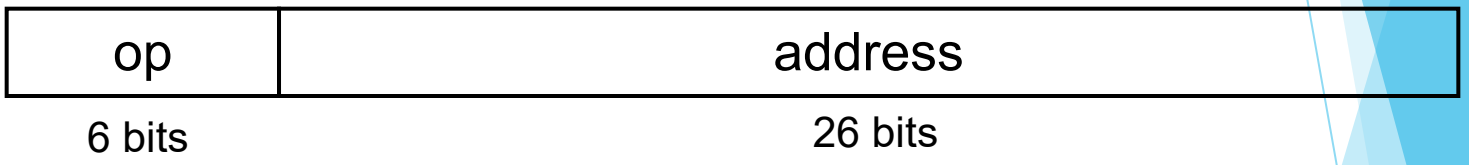


0000 1000 0000 0000 0000 0100 0000 0100 (binary)

0x08000404 (hexadecimal)

Jump Addressing

- Jump (`j` and `jal`) targets could be anywhere in text segment
  - Encode full address in instruction



- (Pseudo)Direct jump addressing
  - Target address = $PC_{31...28}$: (address × 4)

MIPS Machine Language Instructions

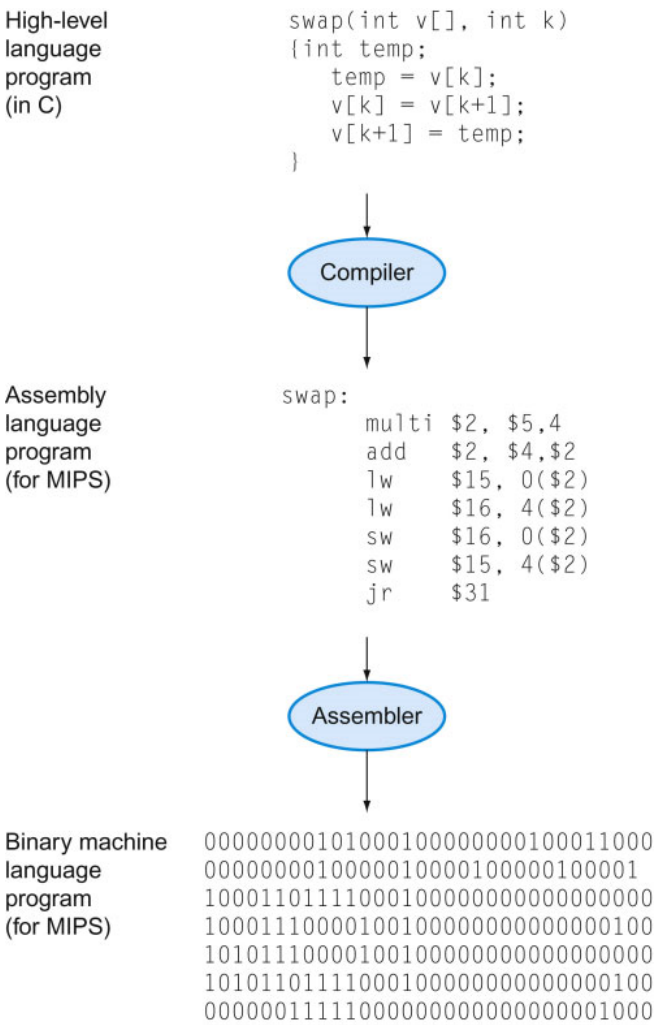
- MIPS **branch** and **jump** instructions.

```
Loop: sll  $t1, $s3, 2
      add  $t1, $t1, $s6
      lw   $t0, 0($t1)
      bne  $t0, $s5, Exit
      addi $s3, $s3, 1
      j    Loop
Exit: …
```

80000	0	0	19	9	4	0
80004	0	9	22	9	0	32
80008	35	9	8	0		
80012	5	8	21	2		
80016	8	19	19	1		
80020	2	20000				
80024						

Program Execution

High-Level, Assembly, and Machine



High-level Language

- Portable
- Can be executed on any processor for which there is a compiler.

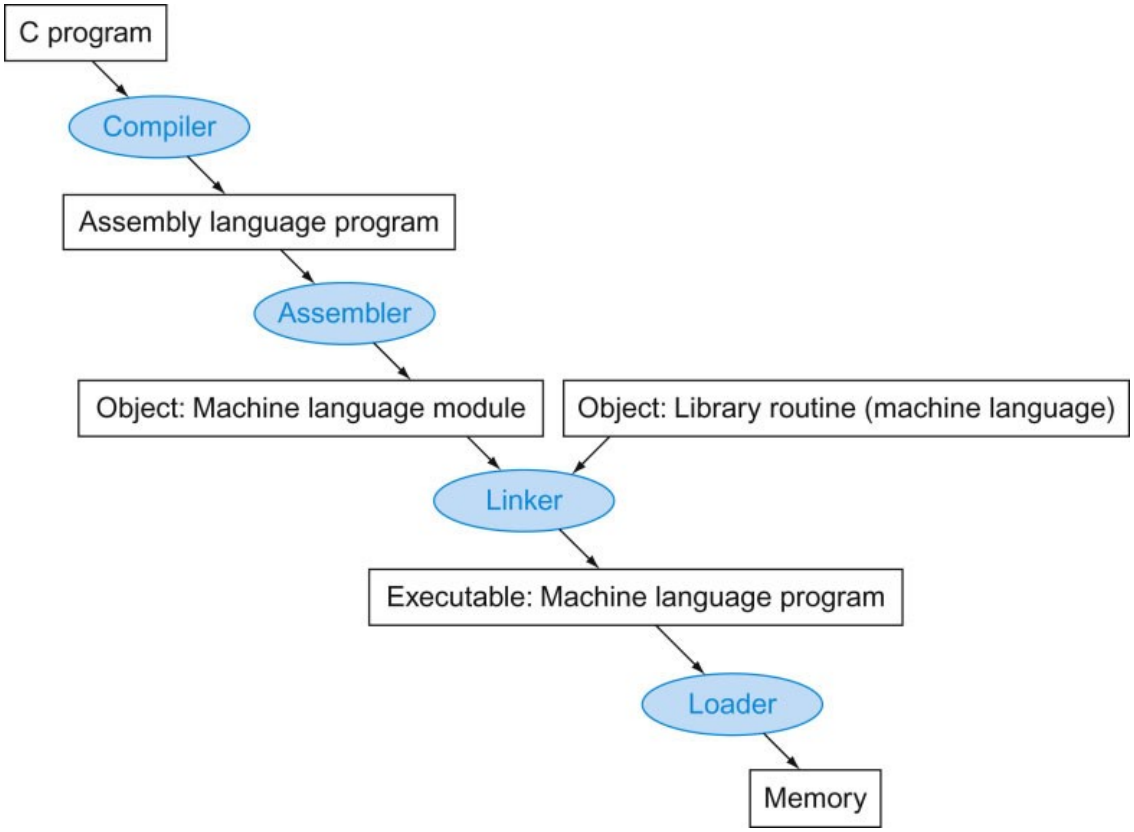
Assembly Language (Low-level)

- Processor-specific
- Cannot be executed on a different processor.
- Human-readable form.

Machine Language (Low-level)

- Processor-specific
- One-to-one correlation with assembly language instructions.
- Machine-readable form.

Creating an Executable Program



Compiler

- Translates HLL program into assembly language.
- Processor-specific.

Assembler

- Translates assembly language into machine language.
- Creates an object file.

Linker

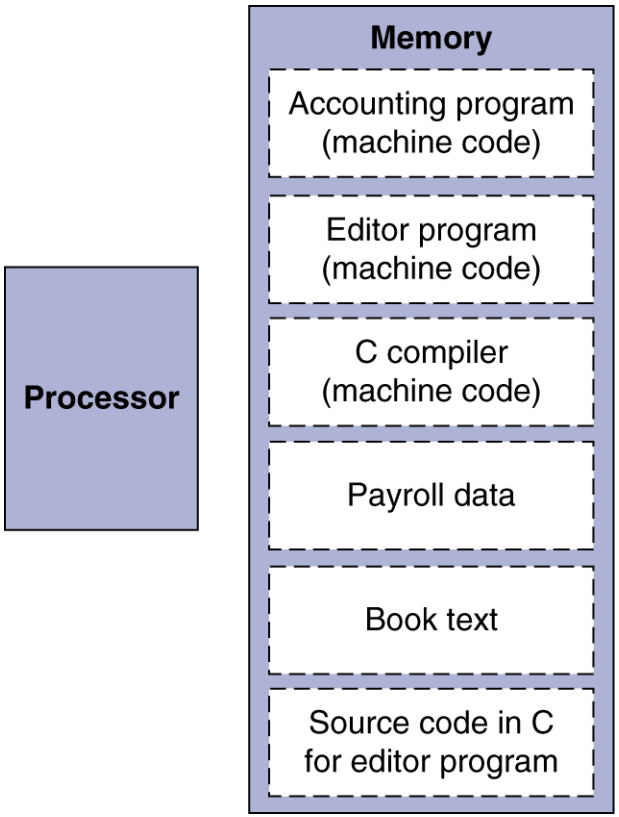
- Combines multiple object files, including library files, into a single executable program.

Loader

- Loads the executable program into memory.
- Prepares it for execution.

Stored Program Computers

The BIG Picture



- Instructions represented in binary, just like data
- Instructions and data stored in memory
- Programs can operate on programs
  - e.g., compilers, linkers, …
- Binary compatibility allows compiled programs to work on different computers
  - Standardized ISAs

Program Execution

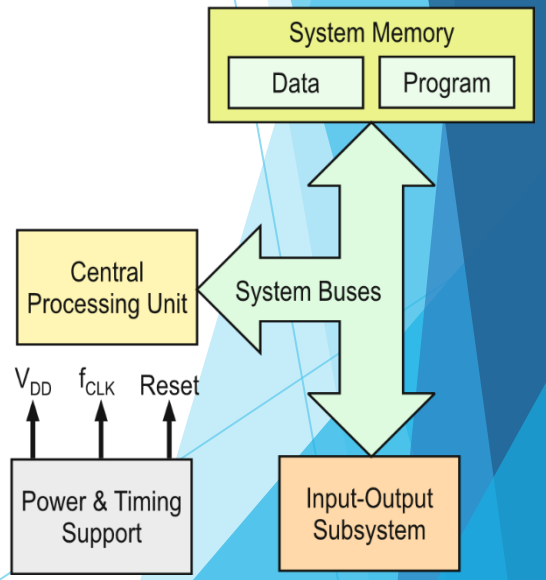
- The processor executes machine language instructions
- Instructions are stored in program memory, each at a specific address.
- Each instruction must be **fetched from memory** before it can be executed by the processor (CPU).

Assembly: User Level

```
LOAD    [10]
ADD     [11]
STORE   [12]
```

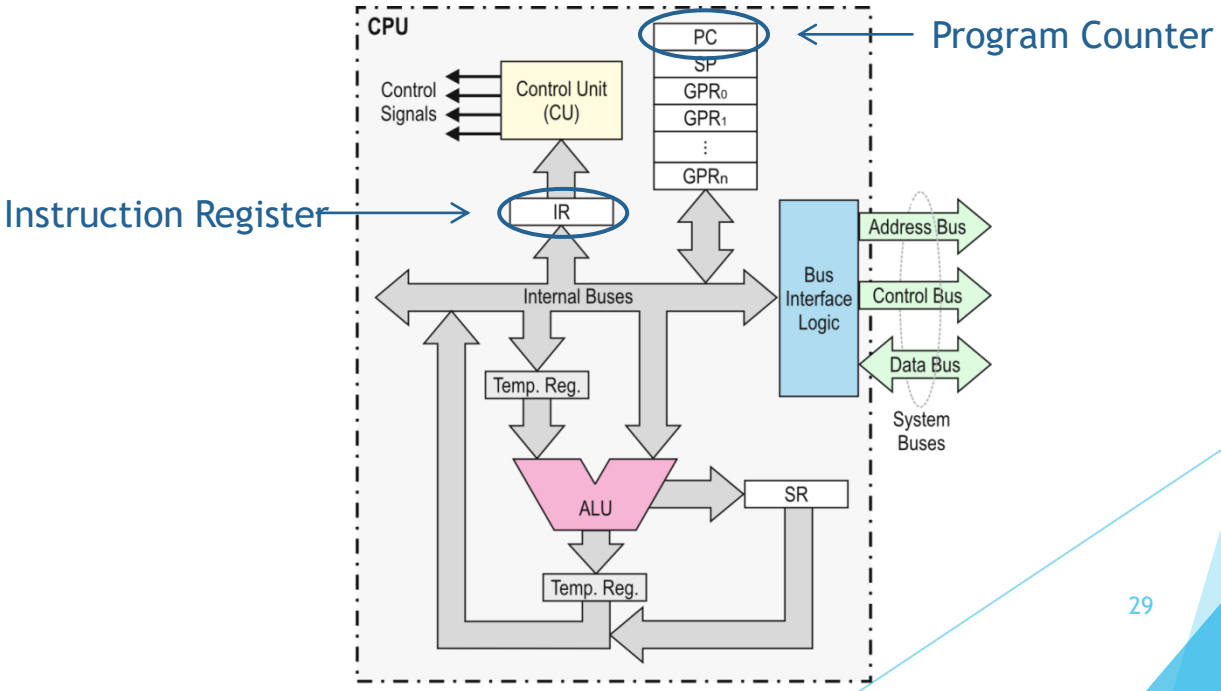
Machine Level

```
100110 0000001010
110011 0000001011
111010 0000001100
```



Program Execution

- The processor executes machine language (ML) instructions.
- Special purpose registers
  - Program Counter (PC): stores the address of the instruction to be fetched.
  - Instruction Register (IR): stores the instruction to be executed by the CPU.

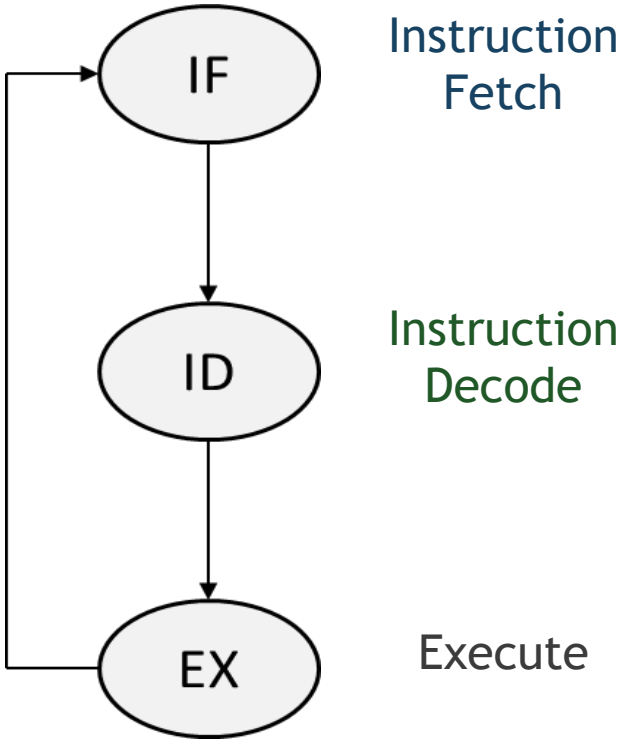


Registers

- Instruction Pointer **(IP)**
  - Stores the location of the **next** instruction
  - Also called the Program Counter **(PC)**
- Memory Address Register **(MAR)**
  - Stores the current instruction address
- Memory Buffer Register **(MBR)**
- Current Instruction Register **(CIR)** or **(IR)**
  - Stores the **current** instruction.
- Accumulator **(AX)**
  - Used for short term storage

Program Execution

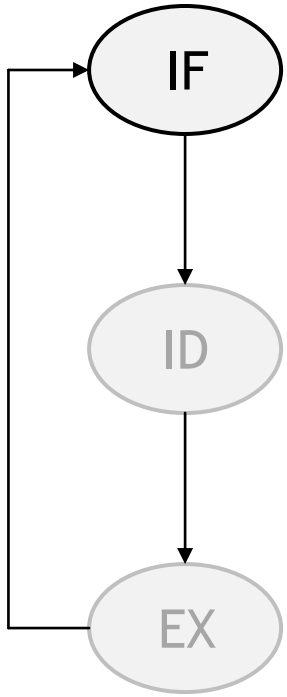
- Each machine language instruction is processed according to the steps in the *instruction cycle*.



Instruction Cycle

Instruction Cycle

- Each machine language instruction is processed according to the steps in the *instruction cycle*.

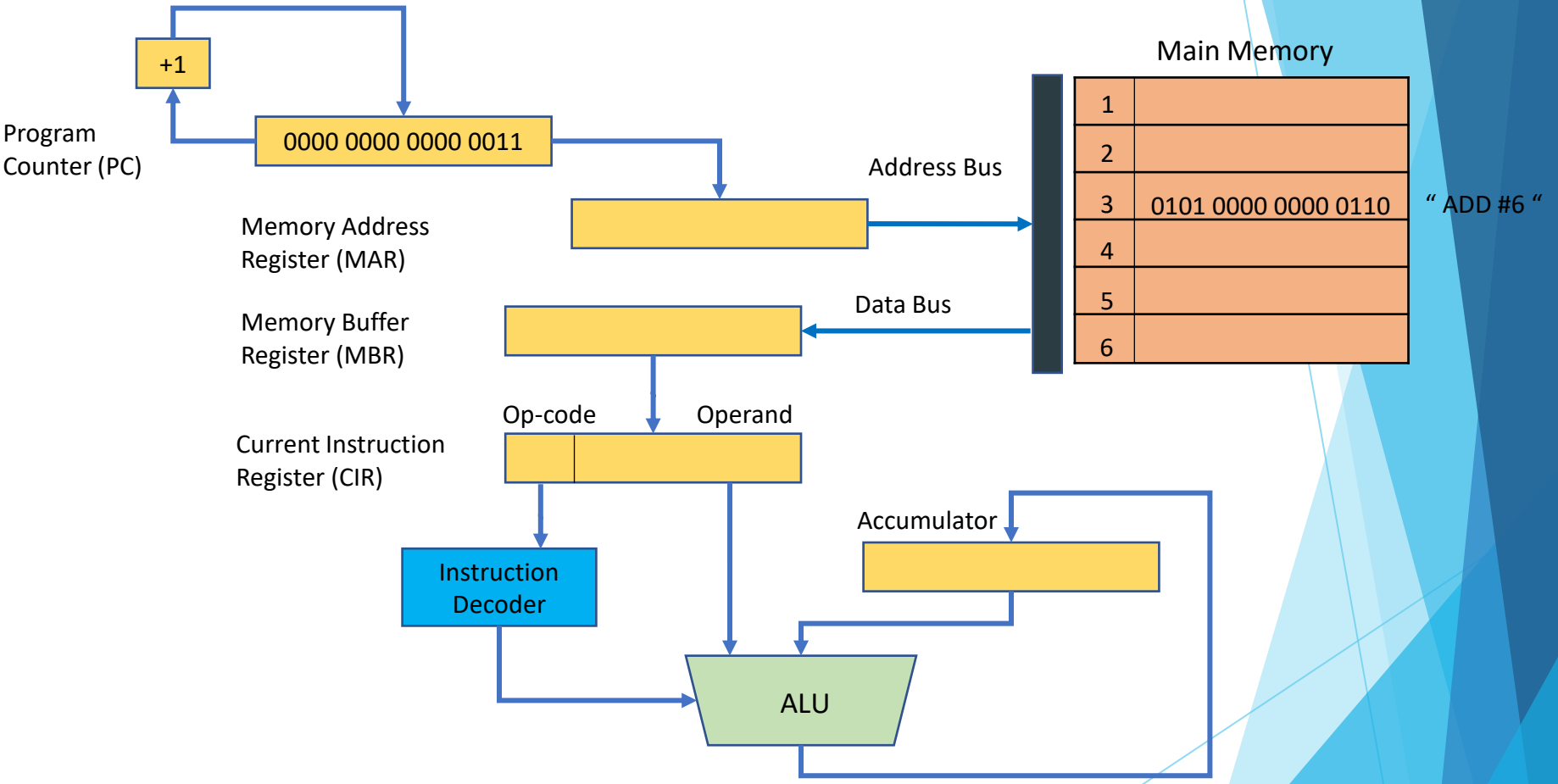


Instruction Cycle

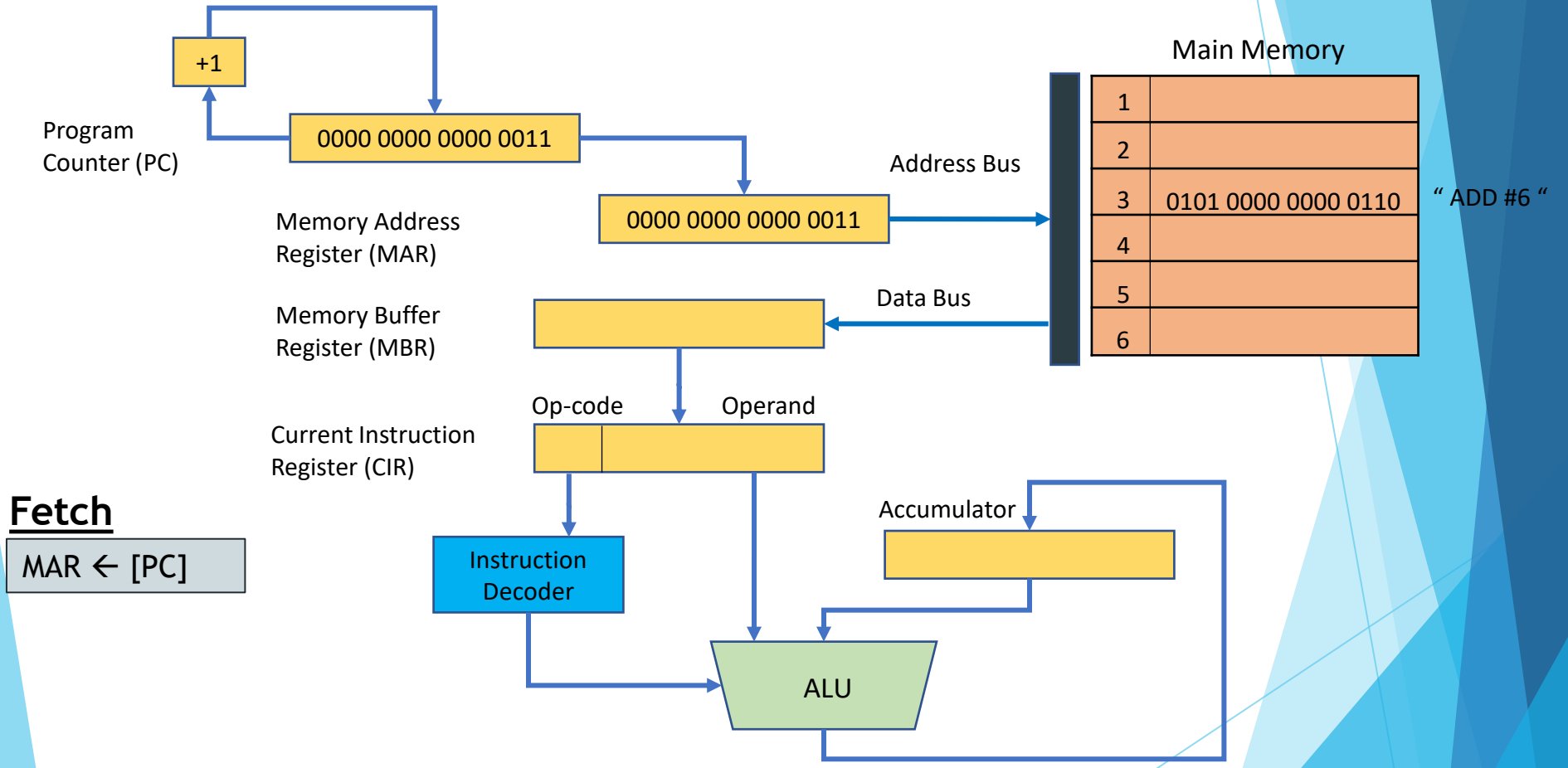
Instruction Fetch

- PC contains address of instruction.
- Read (fetch) instruction from program memory at address pointed to by PC.
- Adjust PC to point to next instruction.
  - Increment PC (by 4 for MIPS)

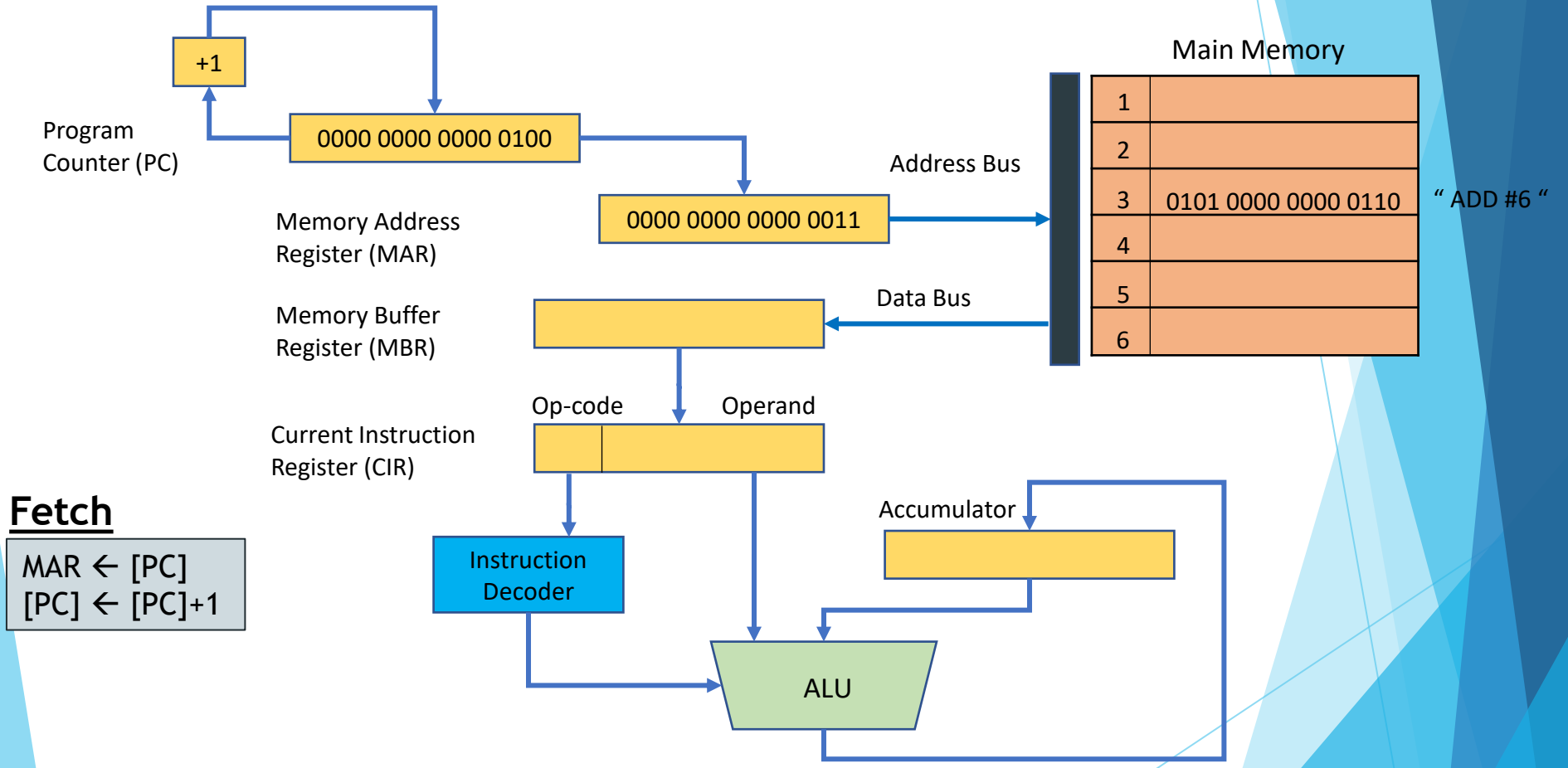
Instruction Cycle



Instruction Cycle



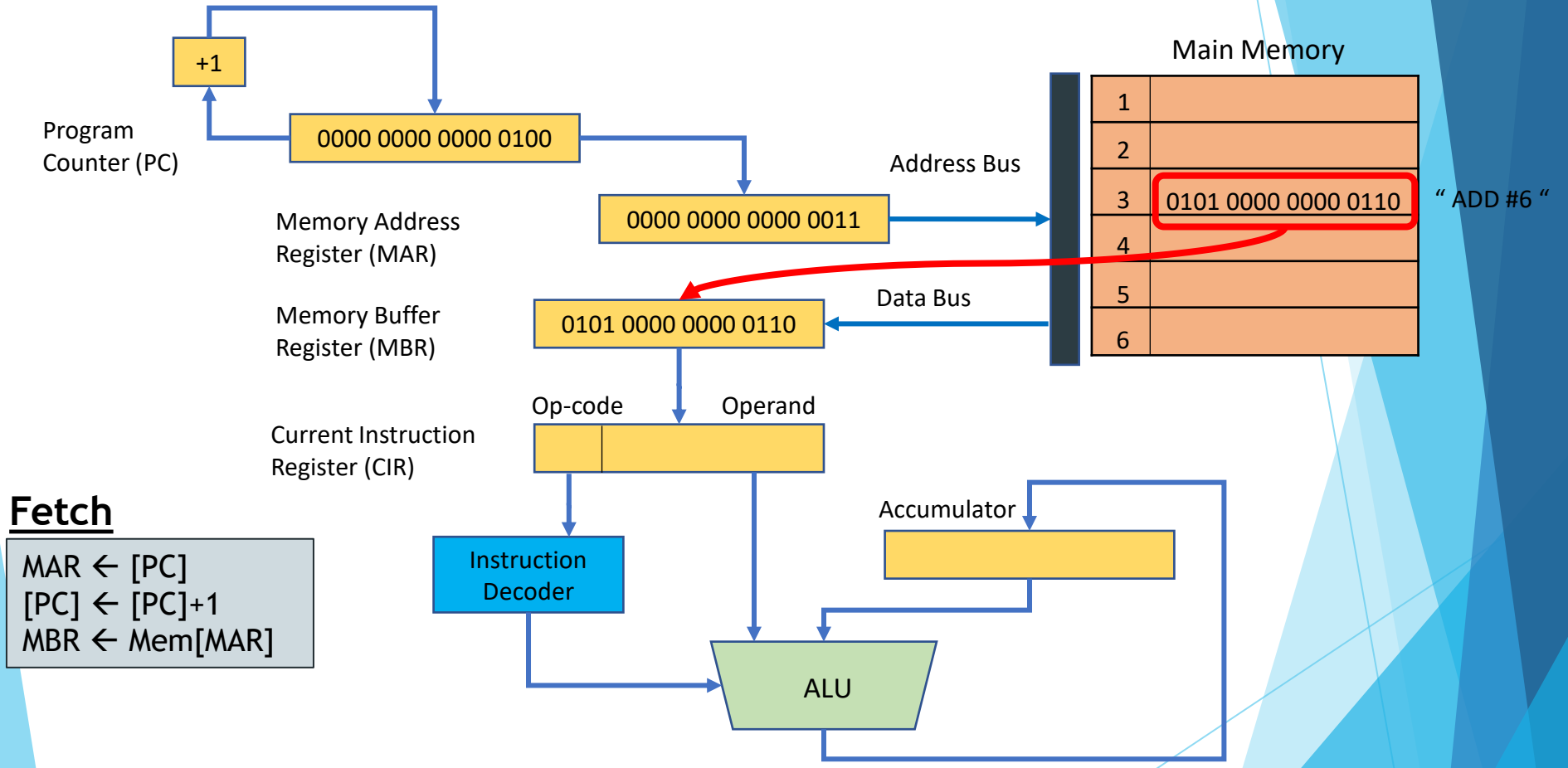
Instruction Cycle



Fetch

MAR ← [PC]
[PC] ← [PC]+1

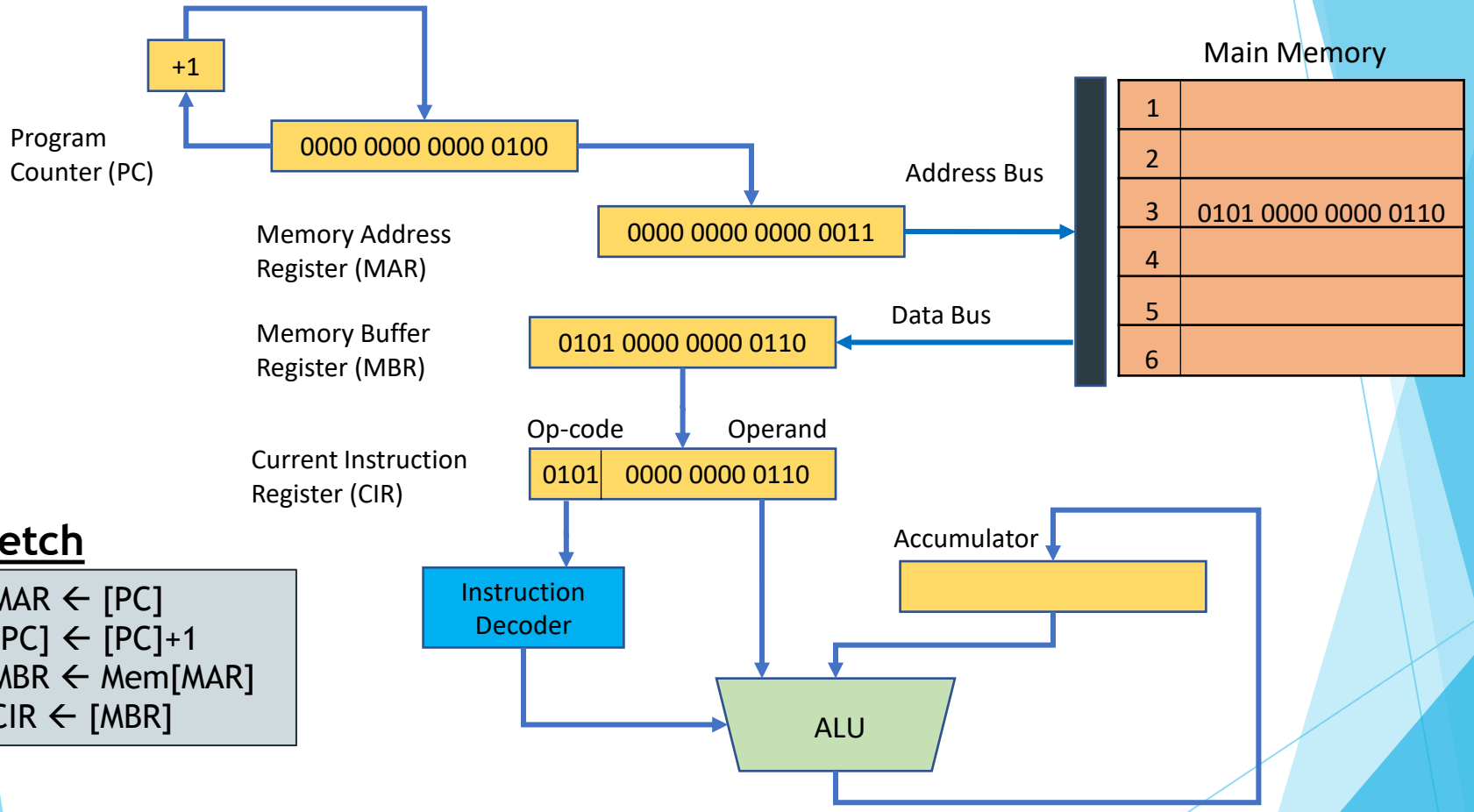
Instruction Cycle



Fetch

MAR ← [PC]
[PC] ← [PC]+1
MBR ← Mem[MAR]

Instruction Cycle

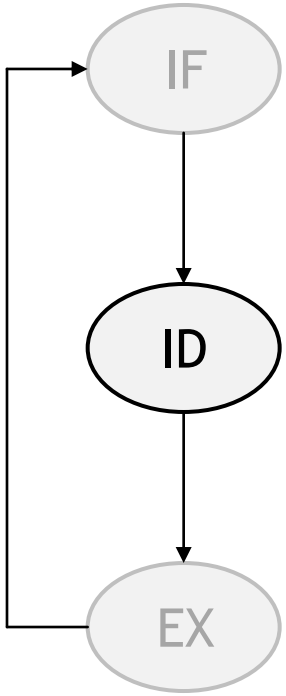


Fetch

MAR ← [PC]
[PC] ← [PC]+1
MBR ← Mem[MAR]
CIR ← [MBR]

Instruction Cycle

- Each machine language instruction is processed according to the steps in the *instruction cycle*.

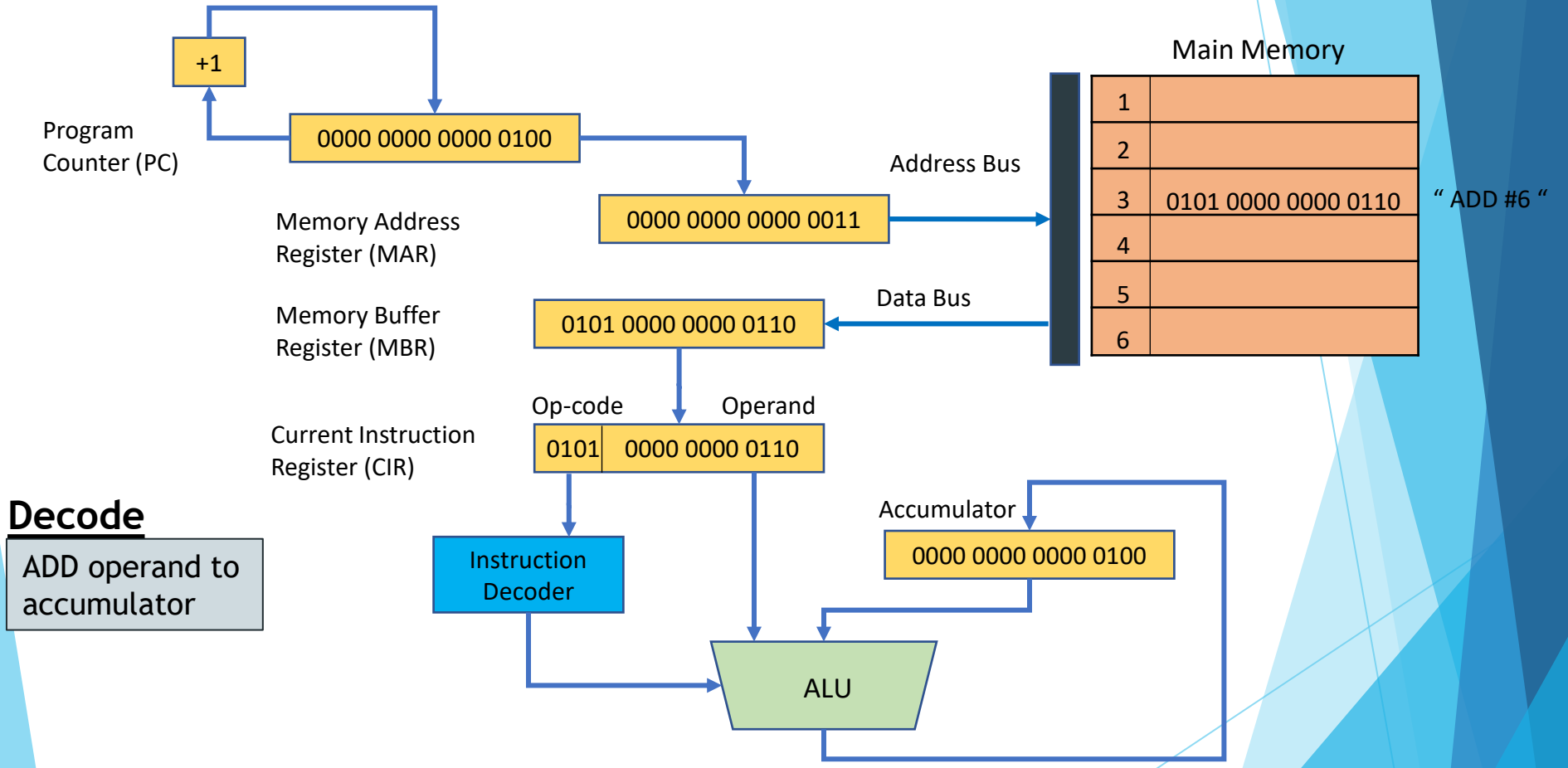


Instruction Cycle

Instruction Decode

- Instruction is identified by the *opcode*
- Control unit decodes instruction
- Appropriate control signals are generated
  - Control the functional units in the datapath needed to execute the instruction.
  - Control unit generates a different set of control signals for different instructions

Instruction Cycle

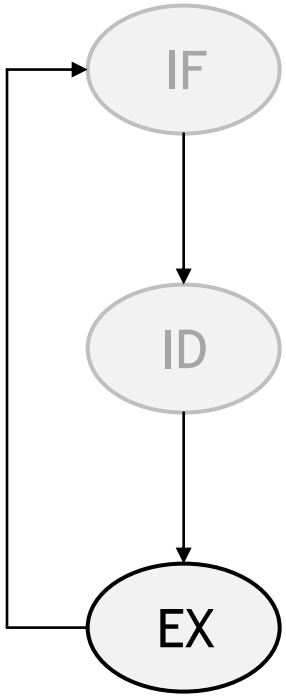


Decode

ADD operand to accumulator

Instruction Cycle

- Each machine language instruction is processed according to the steps in the *instruction cycle*.

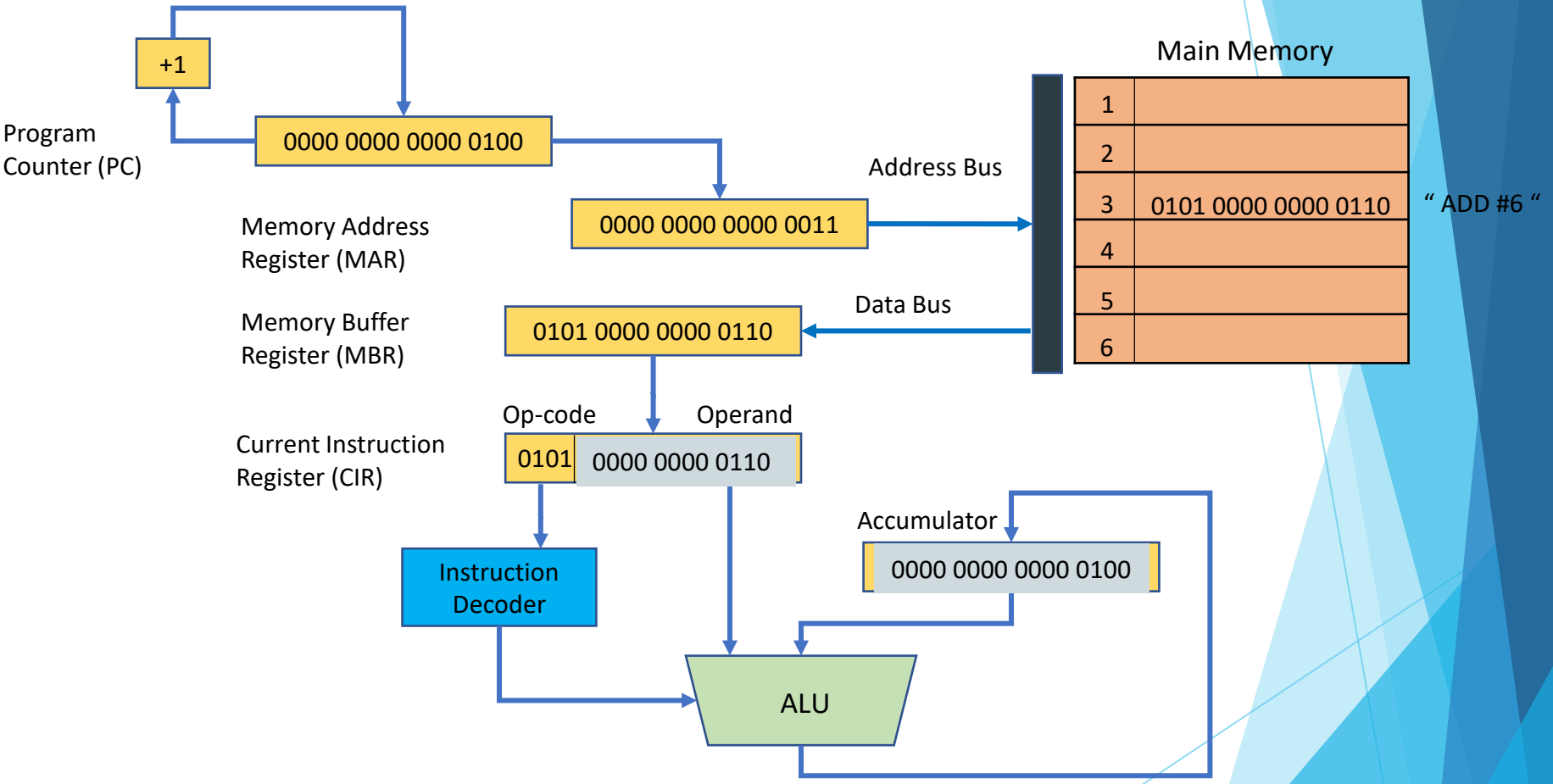


Instruction Cycle

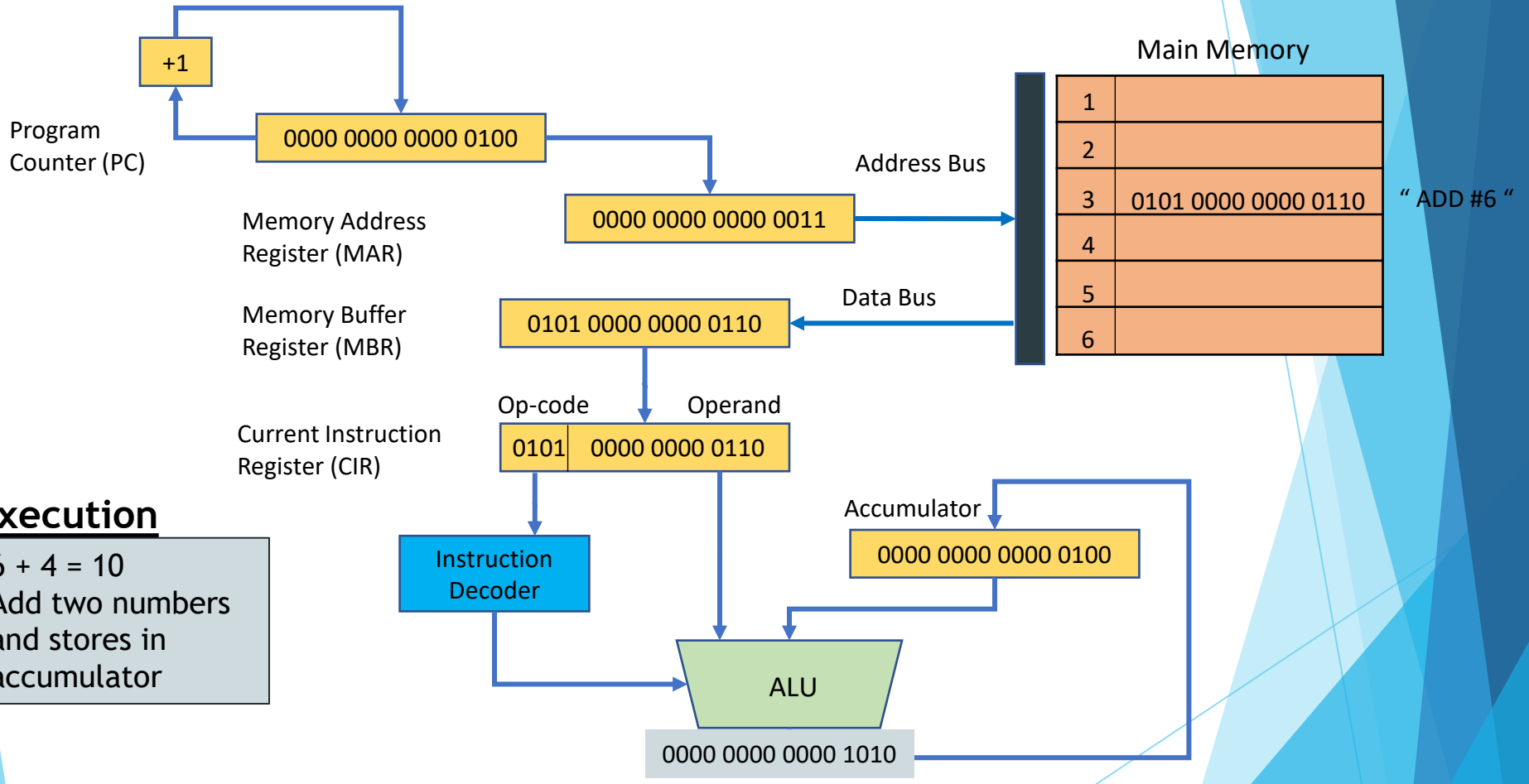
Execute

- Functional units in the datapath implement the required actions for the executed instruction.
  - Functional units controlled by the control signals.
  - Different instructions require a different set of actions and, thus, a different set of functional units.

Instruction Cycle



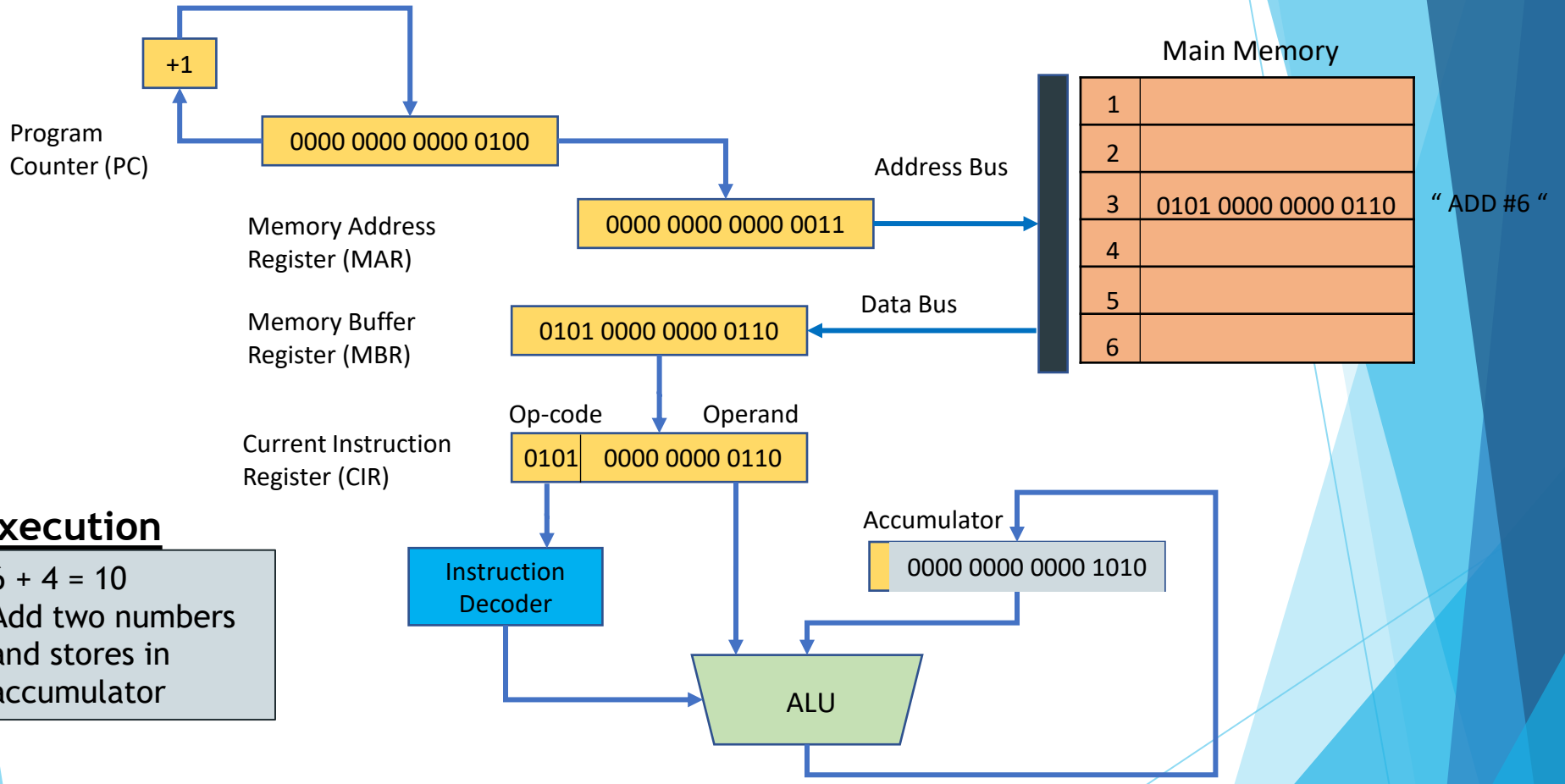
Instruction Cycle



Execution

6 + 4 = 10
Add two numbers and stores in accumulator

Instruction Cycle



Execution

6 + 4 = 10
Add two numbers and stores in accumulator

Instruction Cycle: Recap

- Instruction cycle includes a 3-fold process:
  - **Fetch:** Reading the instruction from memory
  - **Decode:** Find out the functionality of the instruction
  - **Execution:** Perform the functions of the decoded instruction

